Profili differenti per un Assistente AI

Prof. Ing. Loris Penserini, PhD

https://orcid.org/0009-0008-6157-0396

Overview

Parleremo di:

- Introduzione ai modelli LLM in pratica con TensorFlow
- Assistente AI con «Ollama»
- IoT con smart plugs: Shelly & Tapo
- Comandi vocali con «pyttsx3» e Ollama
- UI per Python con QT Designer



Creare un assistente AI con capacità di giocare ruoli diversi

Creare un Assistente AI

La rivoluzione nell'ambito dell'Intelligenza Artificiale è proprio aver creato piattaforme opensource su architetture transformers per creare velocemente Applicazioni basate su reti neurali generative (LLM):

- **Hugging Face** è una delle piattaforme più popolari per il trattamento del linguaggio naturale, e il suo Transformers library è un punto di riferimento nell'ambito dell'NLP.
- **OpenAI** ha creato il modello GPT (Generative Pre-trained Transformer), che ha avuto un impatto enorme sul campo delle AI generative.
- **Rasa** è un framework open-source per la creazione di assistenti virtuali e chatbot intelligenti.
- **T5** è un modello di Google che utilizza un approccio innovativo in cui ogni task di NLP (come la traduzione, il riassunto, ecc.) è trattato come un problema di "text-to-text".
- **Ollama**: Una piattaforma AI che sfrutta modelli generativi basati su transformers.

Ollama

Questi progetti open-source, come Ollama, sono costruiti su architetture transformer che permettono di affrontare una vasta gamma di problemi di linguaggio naturale e interazione conversazionale.

Questi progetti sono altamente personalizzabili e si prestano a essere utilizzati per applicazioni pratiche come assistenti virtuali, chatbot, e sistemi di ricerca avanzati.

La piattaforma Ollama supporta modelli con diverse architetture, tra cui encoder-only, decoder-only e encoder-decoder. Questa flessibilità consente agli utenti di scegliere il modello più adatto alle loro esigenze specifiche.

Ollama

Le similarità tra Ollama e BERT risiedono nel fatto che entrambi:

- Utilizzano transformers come architettura di base.
- Sono progettati per risolvere problemi di comprensione del linguaggio naturale (NLP).
- Si basano su un meccanismo di self-attention che permette al modello di interpretare correttamente il significato contestuale delle parole.

Tuttavia, Ollama potrebbe avere funzionalità più avanzate o specifiche, come l'integrazione con applicazioni pratiche di assistenti virtuali o interazioni dirette con gli utenti, mentre BERT è maggiormente usato per task di comprensione passiva (come l'analisi del sentiment o il riassunto di testi).

Inoltre, **la piattaforma Ollama supporta modelli con diverse architetture, tra cui encoder-only, decoder-only e encoder-decoder.** Questa flessibilità consente agli utenti di scegliere il modello più adatto alle loro esigenze specifiche.

Ollama Installation

Scaricare e installare l'applicazione:

https://www.ollama.com/download

Una volta installato diventa un processo in memoria sempre attivo. In caso occorra rifarlo partire:

C:\Users\...\>**ollama serve**

Per queste lezioni ho installato:

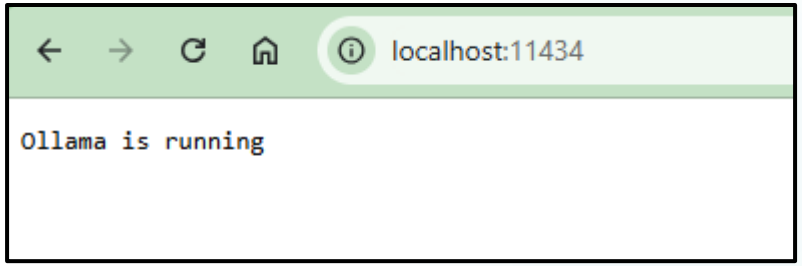
```
C:\Users\admin>ollama -v
ollama version is 0.10.1

C:\Users\admin>ollama list
NAME                ID              SIZE      MODIFIED
llama3.1:latest     46e0c10c039e    4.9 GB    7 weeks ago
```

Ollama Installation

Funziona come un server HTTP:

```
OLLAMA_BASE_URL = "http://localhost:11434"
```



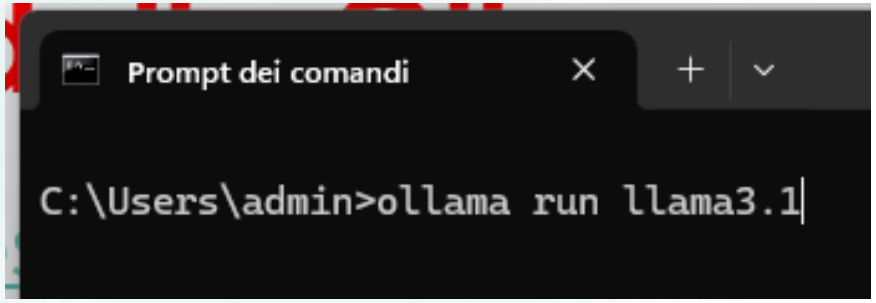
Per vedere i modelli scaricabili per Ollama:

https://github.com/ollama/ollama

Installare un Modello Ollama:

https://ollama.com/library

Per installare un modello basta eseguirlo a riga di comando:



llama3.1

```
ollama run llama3.1
```

99.6M Downloads Updated 8 months ago

Llama 3.1 is a new state-of-the-art model from Meta available in 8B, 70B and 405B parameter sizes.

tools 8b 70b 405b

Models

View all →

Name	Size	Context	Input
llama3.1:latest	4.9GB	128K	Text
llama3.1:8b latest	4.9GB	128K	Text
llama3.1:70b	43GB	128K	Text
llama3.1:405b	243GB	128K	Text

Configurare Python

Prima di invocare l'oracolo occorre installare la libreria:

pip install requests

Le modalità di interazione

Con prompt:

/api/generate

(completion-based)

Con messages:

/api/chat

(chat-based)

RequestResponseAI.py > ...

```python
import requests
import json

# URL dell'API locale di Ollama
url = "http://localhost:11434/api/chat"  # endpoint chat based di Ollama: msg con ruoli
#url = "http://localhost:11434/api/generate" # endpoint a completion-based

# Chiedi all'utente di inserire una domanda o prompt
prompt_input = input("Inserisci la tua domanda: ")

# Interazione in stile "completamento" della frase
'''
payload = {
    "model": "llama3.1",
    "prompt": prompt_input,
    "max_tokens": 100,  # puoi modificare la lunghezza della risposta
    # aggiungi altri parametri se necessari
}
'''

# Interazione in stile ruoli
payload = {
    "model": "llama3.1",
    "messages": [{"role":"user", "content":prompt_input}]
}
```

Prompt o Ruoli?

Con prompt:
/api/generate
(completion-based)

```
# Interazione in stile "completamento" della frase (prompt)
payload = {
    "model": "llama3.1",
    "prompt": prompt_input,
    "max_tokens": 100,  # puoi modificare la lunghezza della risposta
    # aggiungi altri parametri se necessari
}

try:
    response = requests.post(url, json=payload, stream=True)
```

Con messages/ruoli:
/api/chat
(chat-based)

```
# Interazione in stile ruoli
payload = {
    "model": "llama3.1",
    "messages": [{"role":"user", "content":prompt_input}]
}

try:
    response = requests.post(url, json=payload, stream=True)
```

Differenze

Endpoint completion-based:

- Accetta un singolo campo prompt (testo libero).
- Non ha concetto di ruoli né di cronologia della conversazione.
- Adatto per prompt singoli e generazione di testo senza contesto persistente.

Endpoint chat-based:

- Accetta il formato messages con ruoli (system, user, assistant).
- Pensato per conversazioni a più scambi (*multi-turn conversation*), dove il modello riceve più messaggi e mantiene il contesto.

Invocare l'oracolo... ;-)

```
27  try:
28      response = requests.post(url, json=payload, stream=True)
29      response.raise_for_status()  # verifica errori di richieste
30
31      if response.status_code == 200:
32          print("Streaming response from Ollama:")
33          for line in response.iter_lines(decode_unicode=True):
34              if line: #ignora le righe vuote
35                  try:
36                      #parse each line as a JSON object
37                      json_data = json.loads(line)
38                      # Extract and print the assistant's message content
39                      if "message" in json_data and "content" in json_data["message"]:
40                          print(json_data["message"]["content"], end="")
41                  except json.JSONDecodeError:
42                      print(f"\nFailed to parse line:{line}")
43          print() # Assicurati che l'output finisca con una nuova linea
44
45  except requests.exceptions.RequestException as e:
46      print("Errore durante la richiesta:", e)
47
```

Project Work-2: alternativa elegante

Abbiamo fin qui analizzato come invocare Ollama, l'oracolo, nelle due modalità…

In questo project work dovrete trovare un metodo alternativo per invocare l'oracolo, forse più elegante…

PW-2.Soluz.: usiamo il client ufficiale

Sostituiamo completamente la parte **requests.post(...)** e usiamo il client Python ufficiale di Ollama (ollama package):

pip install ollama

Poi utilizzeremo:

response = client.generate(model=model, prompt=prompt_input)

PW-2: Esempio

Implementazione completa su GitHub:

https://github.com/penserini/ITS_AI_Specialist/blob/main/PW_2/RequestOllamaClient.py

RequestOllamaClient.py

```python
# Esempio che utilizza il client ufficiale di Ollama (e non il server web Ollama)
from __future__ import annotations
from typing import Callable, Dict, List, Optional
import ollama

Message = Dict[str, str]  # {"role": "system"|"user"|"assistant", "content": "..."}


class OllamaClient:
    """
    Wrapper class-based per Ollama:
      - generate(): prompt singolo (completion-style)
      - chat(): conversazione completa (chat-style)
      - say(): aggiunge turno utente allo storico e risponde
      - system(): imposta/aggiorna il messaggio di sistema
    Supporta: streaming, callback on_chunk, storico conversazione, opzioni extra.
    """

    def __init__(self, model: str = "llama3.1", system_prompt: Optional[str] = None):
        self.model = model
        self.history: List[Message] = []
        if system_prompt:
            self.history.append({"role": "system", "content": system_prompt})
        self.last_text: str = ""
```

```python
# --- Esempi rapidi ---
if __name__ == "__main__":
    cli = OllamaClient(model="llama3.1", system_prompt="Rispondi in italiano e brevemente.")
    # Prompt singolo, no streaming
    print("== generate (no stream) ==")
    print(cli.generate("Cos'è il versioning semantico in 2 righe?", stream=False))

    # Chat con streaming
    print("\n== chat (stream) ==")
    def on_tok(t: str): print(t, end="", flush=True)
    cli.say("Dammi 3 vantaggi di /api/chat rispetto a /api/generate.", stream=True, on_chunk=on_tok)
    print("\n\n[STORICO] ->", cli.history[-2:])
```

Differenze principali

Aspetto	`requests.post(...)`	`client.generate(...)`
Tipo	Chiamata HTTP manuale	Wrapper Python ufficiale
Streaming	Devi gestirlo tu (`stream=True`, loop su `iter_lines`)	Puoi scegliere `stream=True/False` come argomento, già gestito
Parsing JSON	Devi farlo tu (`json.loads(line)`)	Il client restituisce già oggetti Python (dizionari)
Pulizia codice	Più verboso, più flessibile (puoi fare cose custom)	Più conciso e leggibile
Portabilità	Nessuna dipendenza extra oltre `requests`	Devi installare `ollama` (`pip install ollama`)
Error handling	Devi gestire manualmente `raise_for_status`	Il client ha eccezioni dedicate già pronte

Overview

Parleremo di:

- Introduzione ai modelli LLM in pratica con TensorFlow
- Assistente AI con «Ollama»
- IoT con smart plugs: Shelly & Tapo
- Comandi vocali con «pyttsx3» e Ollama
- UI per Python con QT Designer



Creare un assistente AI con capacità di giocare ruoli diversi

IoT and Smart Plug

IoT (Internet of Things) significa letteralmente "Internet delle Cose": oggetti fisici dotati di elettronica, sensori, connettività di rete e software che possono comunicare tra loro o con servizi cloud.

Le smart plug sono un esempio tipico: prese elettriche intelligenti controllabili da app, comandi vocali o automazioni.

Esempi pratici con TAPO P110 e Shelly…

TAPO P110 vs Shelly Plug type F

Adottano due politiche completamente diverse:

- **TAPO P110** solo comunicazione TP-Link, usa un approccio «chiuso» agli standard
- **Shelly Plug** API http con possibilità di comandi REST

TAPO si sta dirigendo verso lo standard «Matter» per esempio con P110M: la versione Matter compatibile. Lo stesso vale per Shelly con Shelly Plug S MTR Gen3.

Standard «Matter»

Il supporto Matter significa che un dispositivo IoT (come una smart plug, una lampadina o un sensore) è compatibile con Matter, uno **standard di comunicazione unificato pensato per far parlare tra loro dispositivi di marche diverse in modo semplice, sicuro e senza lock-in di ecosistema**.

È uno standard aperto creato dalla Connectivity Standards Alliance (CSA) (ex Zigbee Alliance).

Supportato da Apple, Google, Amazon, Samsung e molti altri.

Funziona su protocolli di rete già esistenti come Wi-Fi ed Ethernet, e su protocolli low-power come Thread.

TAPO P110

Pro

- App Tapo ufficiale per installazione, controllo remoto, e automazioni.
- Supporto Matter nei modelli P110M; in futuro migliorerà l'integrazione con Home Assistant.

Contro

- Nessuna API locale documentata: non puoi controllarla direttamente via REST o MQTT in rete locale.
- Comunicazione esclusivamente via cloud TP-Link, limitando personalizzazione e introducendo dipendenze esterne e possibili lag.
- Alcuni utenti lamentano blocchi dopo aggiornamenti firmware, con errori "unknown API -1" nell'integrazione con Home Assistant.

Shelly Plug Type F

Pro

- API HTTP locale completa: accessibile direttamente via http://<ip>/relay/0?turn=on o /status, senza necessità del cloud.
- Supporto per metering energetico integrato (power meter), utile per monitoraggio consumi.
- Funziona in modalità standalone Wi-Fi e può essere integrata nella tua rete domestica senza dipendenze esterne.
- Modelli moderni supportano MQTT, Webhooks, scripting, e anche Matter (es. Shelly Plug S MTR Gen3).

Contro

- Richiede configurazione e gestione in locale, il che può essere più tecnico rispetto alla Tapo, ma offre maggiore flessibilità.

La «scelta»: filosofie a confronto

Per scegliere la smart plug più adatta è importante conoscere le differenze tra i **protocolli di comunicazione più usati come REST e MQTT**.

REST (Representational State Transfer)

Protocollo di comunicazione request/response basato su HTTP:

- Il client invia una richiesta HTTP (GET, POST, PUT, DELETE) a un server.
- Il server risponde con i dati (in genere JSON).
- Esempi di comandi con smart plug Shelly tipo F:

  http://192.168.1.52/relay/0?turn=off

  http://192.168.1.52/relay/0?turn=on

  http://192.168.1.52 /rpc/Shelly.GetDeviceInfo

  http://192.168.1.52 /rpc/Shelly.GetStatus

  ecc. per altri comandi visitate questo link

MQTT (Message Queuing Telemetry Transport)

Protocollo di messaggistica pub/sub (publish/subscribe) molto leggero.

- C'è un broker centrale (es. Mosquitto).
- I dispositivi pubblicano messaggi su un "topic" (es. casa/soggiorno/presa1/stato).
- Altri dispositivi/servizi si iscrivono a quei topic per ricevere i messaggi in tempo reale.
- Esempio:

  # pubblicare

  topic: casa/soggiorno/presa1/set

  payload: ON

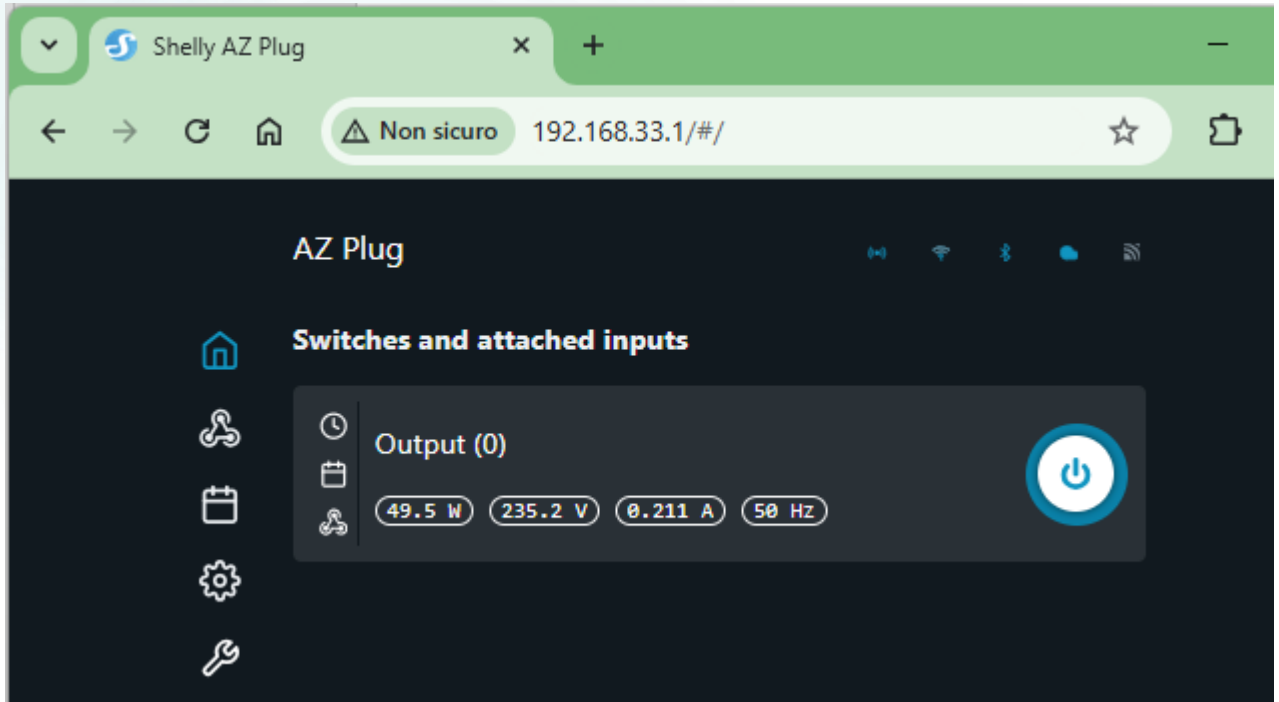
Riassumendo:

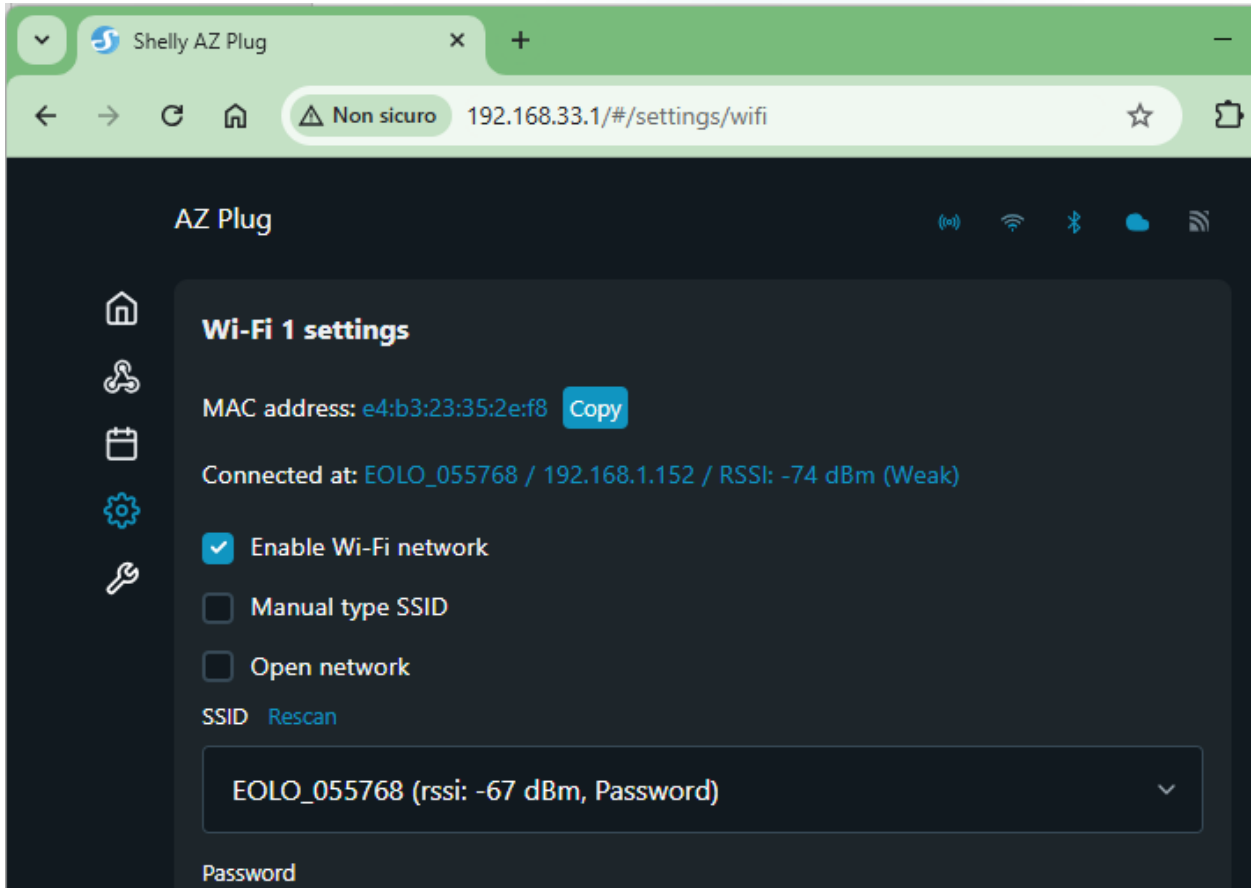
- Push-based → il dispositivo invia aggiornamenti appena cambia stato.
- Consuma pochissima banda → ideale per IoT con connessioni lente o instabili.
- Richiede l'installazione di un broker MQTT attivo nella rete (es. Eclipse Mosquitto).

Preparare la Shelly Plug

La presa può essere configurata tramite una UI via server web accessibile da un browser al link: **192.168.33.1** dopo che ci si è collegati all'access-point della Shelly Plug.

La presa continua ad essere collegata alla rete di casa, mostrando il suo IP.





Preparare la TAPO P110

Si trovano delle librerie da scaricare da questo sito: https://pypi.org/project/tapo/

Trattasi di un unofficial **Tapo API Client** che funziona con TP-Link Tapo smart devices. Inoltre, dovrebbe essere compatibile con:

- light bulbs (L510, L520, L530, L535, L610, L630),
- light strips (L900, L920, L930),
- plugs (P100, P105, P110, P110M, P115),
- power strips (P300, P304),
- hubs (H100) e switches (S200B)
- sensors (KE100, T100, T110, T300, T310, T315).

Poi, ovviamente: **pip install tapo**

>> pip install plugp100

>> pip install kasa

Preparare la TAPO P110

Si trovano delle librerie da scaricare da questo sito: https://pypi.org/project/tapo/

Trattasi di un unofficial **Tapo API Client** che funziona con TP-Link Tapo smart devices. Inoltre, dovrebbe essere compatibile con:

- light bulbs (L510, L520, L530, L535, L610, L630),
- light strips (L900, L920, L930),
- plugs (P100, P105, P110, P110M, P115),
- power strips (P300, P304),
- hubs (H100) e switches (S200B)
- sensors (KE100, T100, T110, T300, T310, T315).

Poi, ovviamente: **pip install tapo**

>> pip install plugp100 (https://pypi.org/project/plugp100/)

>> pip install kasa (https://pypi.org/project/python-kasa/)

Project Work-3

Realizzare un Client, che si interfacci con API TAPO, per attivare e disattivare la smart plug P110.

Soluz. PW-3

Esempio completo su GitHub:

https://github.com/penserini/ITS_AI_Specialist/tree/main/PW_3

simpleTestFree.py

```python
"""P110 and P115 Example"""
import asyncio
import os
from datetime import datetime

from tapo import ApiClient
from tapo.requests import EnergyDataInterval


async def main():

    os.environ['TAPO_USERNAME'] = " **** "
    os.environ['TAPO_PASSWORD'] = " **** "
    os.environ['IP_ADDRESS'] = " **** "

    tapo_username = os.getenv("TAPO_USERNAME")
    tapo_password = os.getenv("TAPO_PASSWORD")
    ip_address = os.getenv("IP_ADDRESS")

    client = ApiClient(tapo_username, tapo_password)
    device = await client.p110(ip_address)

    print("Turning device on...")
    await device.on()

    print("Waiting 2 seconds...")
    await asyncio.sleep(2)

    print("Turning device off...")
    await device.off()
```

Overview

Parleremo di:

- Introduzione ai modelli LLM in pratica con TensorFlow
- Assistente AI con «Ollama»
- IoT con smart plugs: Shelly & Tapo
- Comandi vocali con «pyttsx3» e Ollama
- UI per Python con QT Designer



Creare un assistente AI con capacità di giocare ruoli diversi

Comandi vocali e LLaMA

Riconoscimento vocale (Speech-to-Text)

Serve per convertire la voce in testo. Puoi usare:

- **Whisper (OpenAI)** – open source e molto preciso
- Vosk – leggero e offline
- Google Speech API, Mozilla DeepSpeech – alternative

Elaborazione NLP (con LLaMA)

Puoi usare LLaMA via Ollama per "capire" il comando e decidere cosa fare.

Esempio: 'accendi la luce', poi LLaMA elabora e genera un'azione o una risposta.

Output (azione o risposta vocale)

Per risposte vocali usa **Text-to-Speech (TTS)** come: espeak, gTTS, Coqui TTS, Google TTS.

Per azioni domotiche e integrazione con sistemi tipo: Home Assistant, MQTT, HTTP API locali (es. Shelly).

Librerie «Speech-to-Text»

Per questi esempi è stato utilizzato Whisper che è un modello di riconoscimento vocale di uso generale. È addestrato su un ampio set di dati audio eterogenei ed è anche un modello multitasking in grado di eseguire il riconoscimento vocale multilingue, la traduzione vocale e l'identificazione della lingua.

pip install -U openai-whisper

Per aggiornare il package all'ultima versione inserita su github:

pip install --upgrade --no-deps --force-reinstall git+https://github.com/openai/whisper.git

It also requires the command-line tool **ffmpeg** to be installed on your system, which is available from most package managers:

Dettagli disponibili qui: https://github.com/openai/whisper

Poi è stato installato:

pip install sounddevice scipy

Librerie «Speech-to-Text»

Poi è stato installato:

pip install sounddevice scipy

sounddevice è una libreria Python che fornisce un'interfaccia per la registrazione e la riproduzione dell'audio attraverso i dispositivi di input e output audio del tuo sistema (come microfono e altoparlanti). È particolarmente utile per l'interazione con l'audio in tempo reale.

scipy è una libreria fondamentale per il calcolo scientifico in Python. Offre una vasta gamma di funzioni per operazioni numeriche e scientifiche, tra cui l'elaborazione dei segnali audio, l'analisi e la manipolazione dei dati.

Librerie per «Text-to-speech»

espeak: disponibile solo per Linux/macOS

pyttsx3: **funziona anche su Windows**

Con Python:

pip install pyttsx3

```
import pyttsx3
def parla(testo):
    engine = pyttsx3.init()
    engine.say(testo)
    engine.runAndWait()
```

Project Work-4

Realizzare un semplice Assistente AI esperto di Dispositivi IoT.

L'Assistente deve interfacciarsi con una smart-plug Shelly, collegata ad una lampada, e comprendere i desiderata dell'utente per accendere o spegnere la lampada. L'utente invia la query da terminale.

Soluz. PW-4

Esempio completo su GitHub:

https://github.com/penserini/ITS_AI_Specialist/tree/main/PW_4

voiceShelly2.py × Settings

voiceShelly2.py > AIbaby > parla

```python
import whisper
import requests
import os
import subprocess
import sounddevice as sd
from scipy.io.wavfile import write
import pyttsx3

class AIbaby(object):
    #testo = ""

    #@staticmethod
    def parla(testo):
        engine = pyttsx3.init()
        engine.say(testo)
        engine.runAndWait()

    # Funzione per registrare audio
    def registra_audio(file_name="audio.wav", durata=5, fs=16000):
        print("Registrazione in corso...")
        registrazione = sd.rec(int(durata * fs), samplerate=fs, channels=1, dtype='int16')
        sd.wait()
        write(file_name, fs, registrazione)
        print("Registrazione salvata.")

    # Funzione per trascrivere con Whisper
    def trascrivi_audio(file_name="audio.wav"):
        model = whisper.load_model("base")
        result = model.transcribe(file_name)
        print(f"Hai detto: {result['text']}")
        return result['text']
```

Soluz. PW-4: KB dell'assistente

```python
# === FLUSSO PRINCIPALE ===
if __name__ == "__main__":
    while True:
        app = AIbaby
        app.registra_audio()
        testo = app.trascrivi_audio()
        prompt = f"""
                L'utente ha detto: "{testo.strip()}".
                Il comando che l'utente sta cercando di eseguire è uno dei seguenti: 'accendi', 'spegni', 'esci'.
                Rispondi solo con una di queste parole, se la risposta non è chiara, rispondi con 'niente'.
                """
        azione = app.interpreta_testo(prompt)
        azione = azione.replace("'",'')
        azione = azione.replace(".",'')
        if azione == "esci" or azione == 'esci':
            break

        app.esegui_azione(azione)
```

```python
    # Funzione per inviare input a LLaMA (via Ollama REST API)
    def interpreta_testo(prompt):
        print("*******",prompt)

        response = requests.post(
            "http://localhost:11434/api/generate",
            json={"model": "llama3.1", "prompt": prompt, "stream": False}
        )
        risposta = response.json()["response"].strip().lower()
        print(f"🤖 LLaMA ha risposto: {risposta}")
        return risposta
```

Soluz. PW-4: «azioni»

```python
    # Funzione per eseguire un'azione
    def esegui_azione(azione):
        azione = azione.replace("'",'')
        azione = azione.replace(".",'')
        shelly_ip = "192.168.1.152"
        if azione == "accendi" or azione == 'accendi':
            #print("Invio comando: accendi la luce")
            try:
                requests.get(f"http://{shelly_ip}/relay/0?turn=on", timeout=3)
                AIbaby.parla("Ho acceso la luce.")
            except:
                AIbaby.parla("Errore: non riesco a raggiungere la presa.")
        elif azione == "spegni" or azione == 'spegni':
            #print("Invio comando: spegni la luce")
            try:
                requests.get(f"http://{shelly_ip}/relay/0?turn=off", timeout=3)
                AIbaby.parla("Ho spento la luce.")
            except:
                AIbaby.parla("Errore: non riesco a raggiungere la presa.")
        elif azione == "esci" or azione == 'esci':
            print("👋 Richiesta di uscire.")
            AIbaby.parla("Arrivederci!")
        else:
            print("🤷 Nessuna azione riconosciuta.")
            AIbaby.parla("Non ho capito il comando.")
```

Bibliografia

[Raschka et al., book 2021] Sebastian Raschka, Vahid Mirjalili, «Machine Learning con Python – Nuova edizione – Costruire algoritmi per generare conoscenza», Apogeo. ISBN: 978-88-503-3524-4. 2021.

[Lambert, book 2024] Kenneth A. Lambert, «Programmazione in Python – Terza edizione», Apogeo education – Maggioli Editore. ISBN: 978-88-916-7143-1. 2024.

[Penserini, GitHub 2025] Loris Penserini, «Corso ITS Academy Turismo Marche: AI Specialist», https://github.com/penserini/ITS_AI_Specialist, 2025.